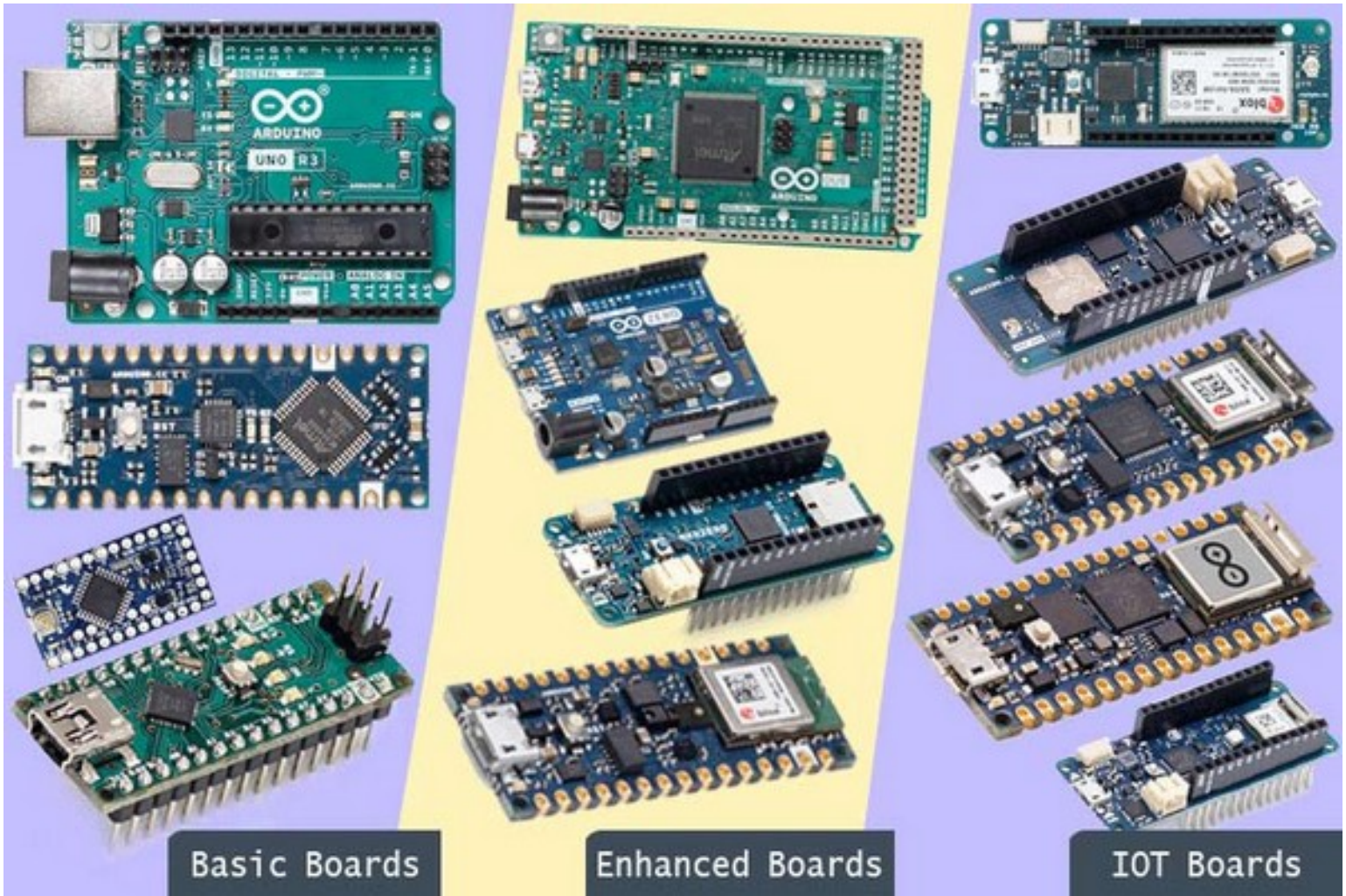
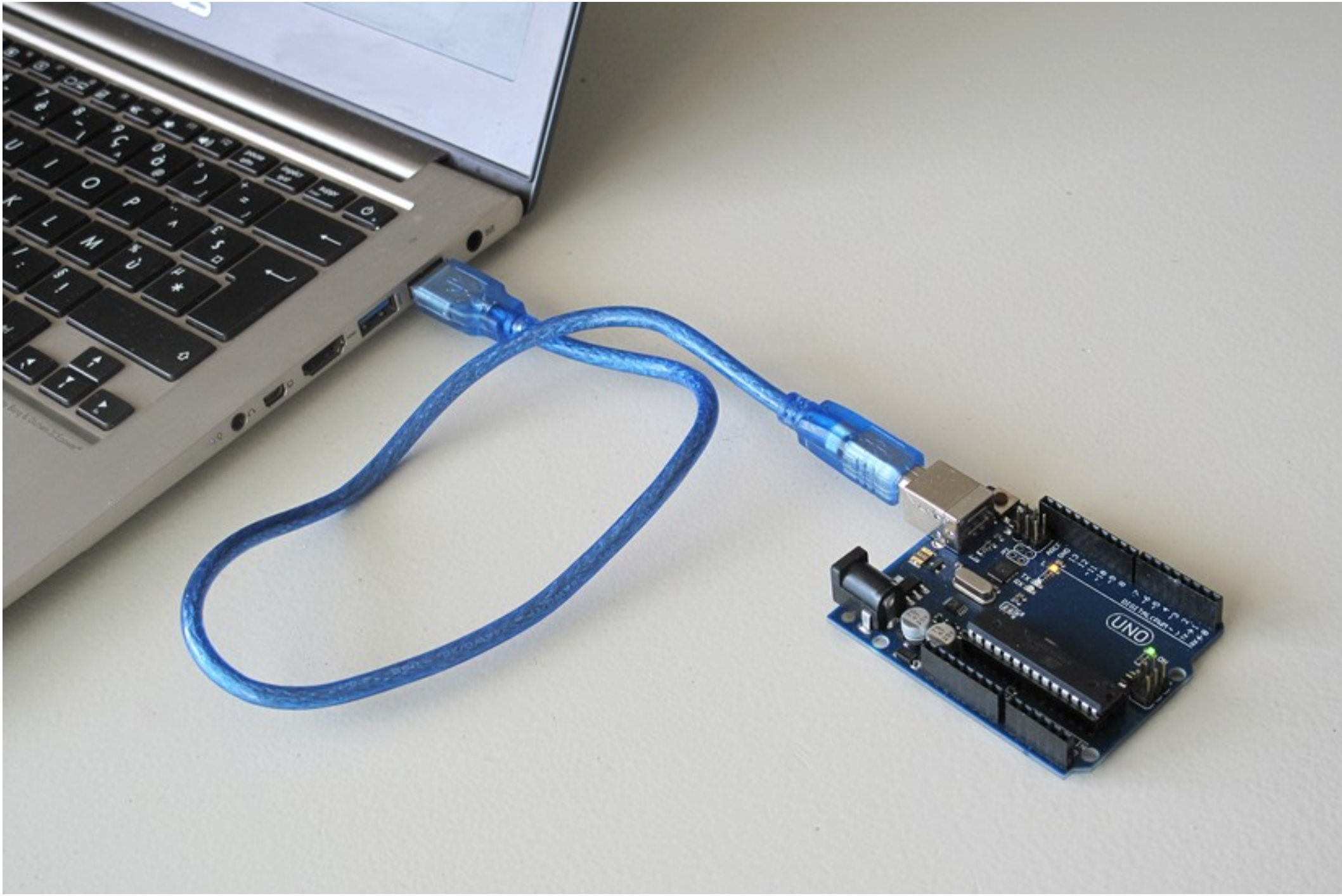
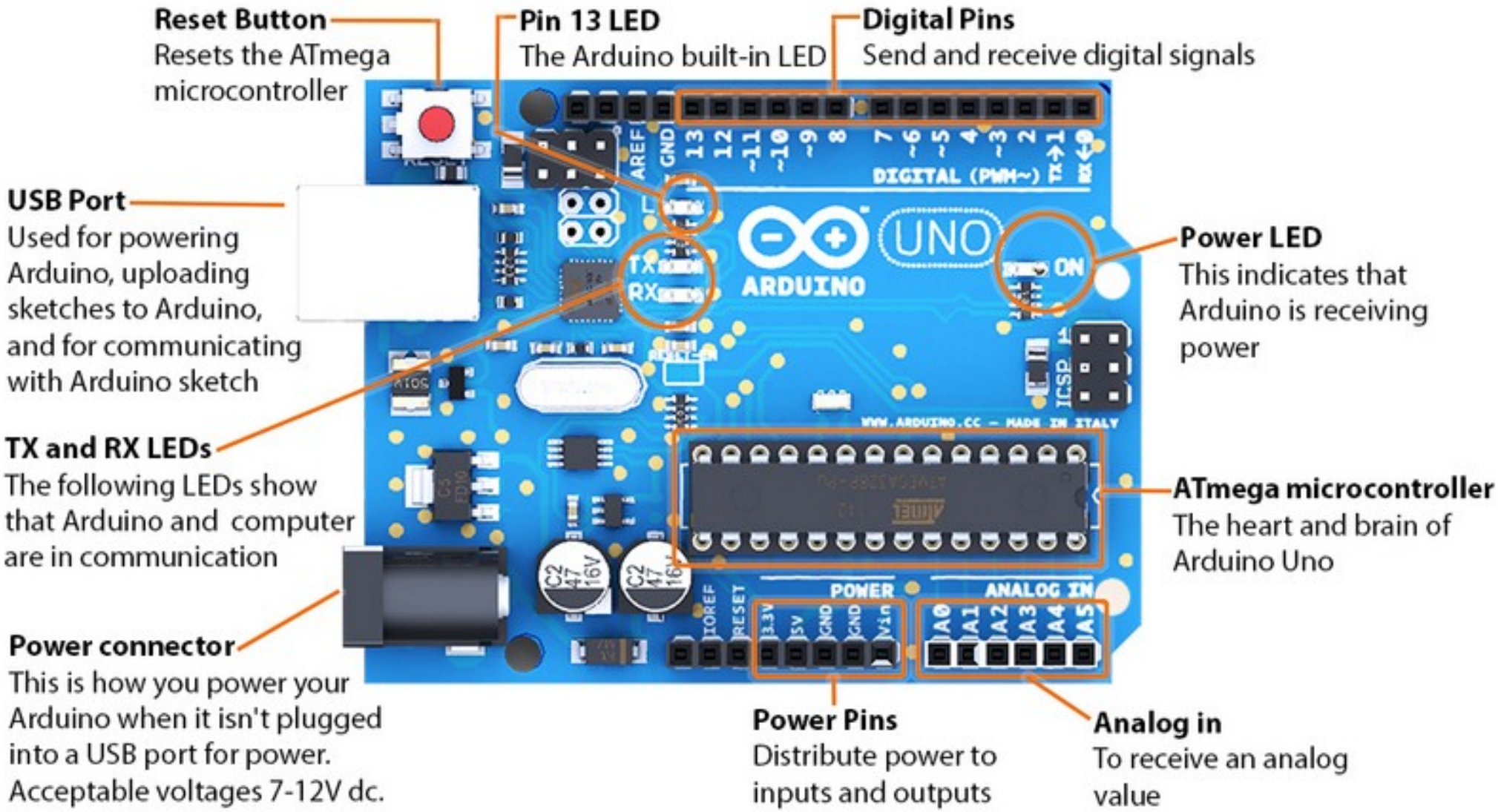
Microcontrolador: Arduino

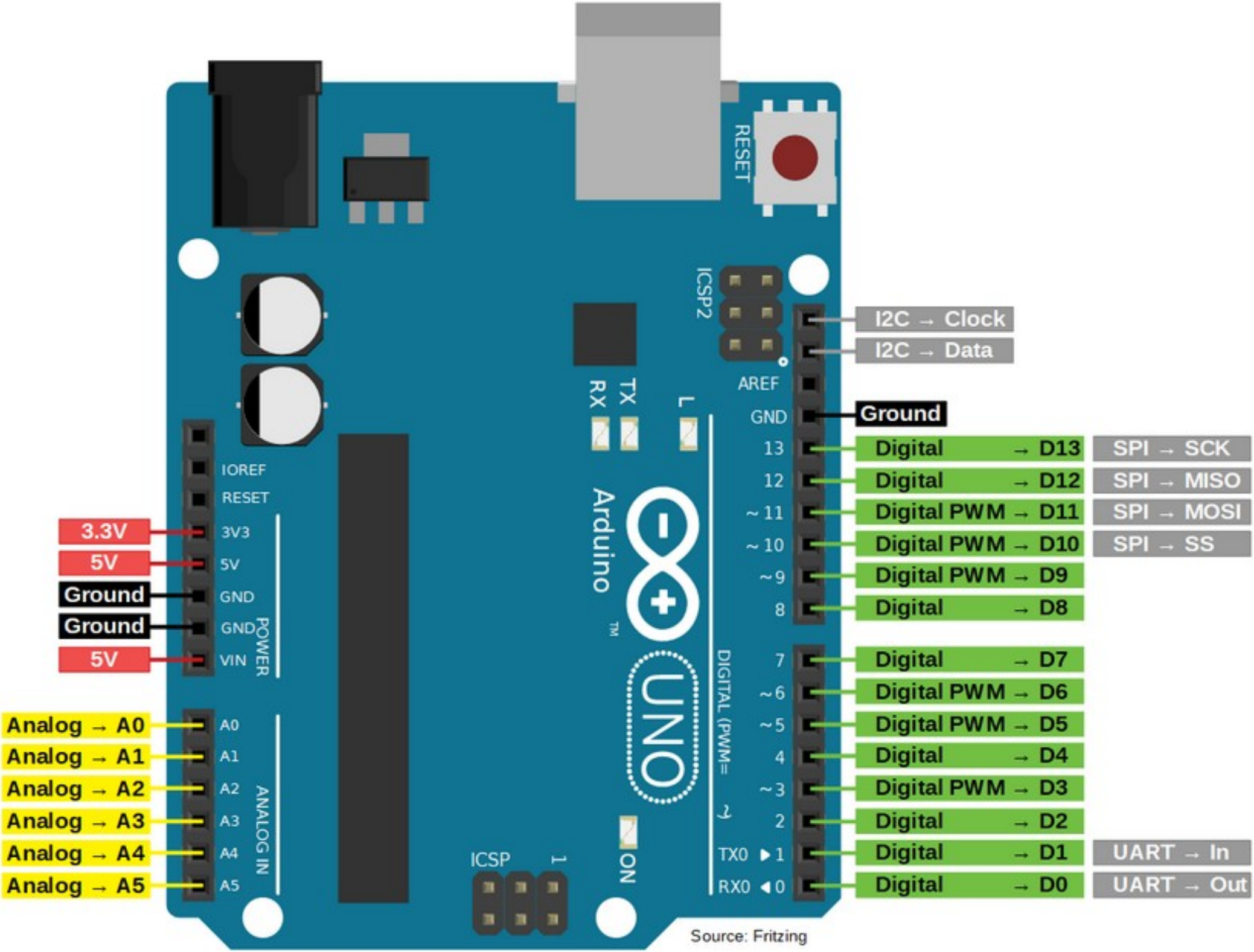
Arduino

Arduino é uma empresa de hardware e software de código aberto, projeto e comunidade de usuários que projeta e fabrica microcontroladores de placa única e kits de microcontroladores para a construção de dispositivos digitais



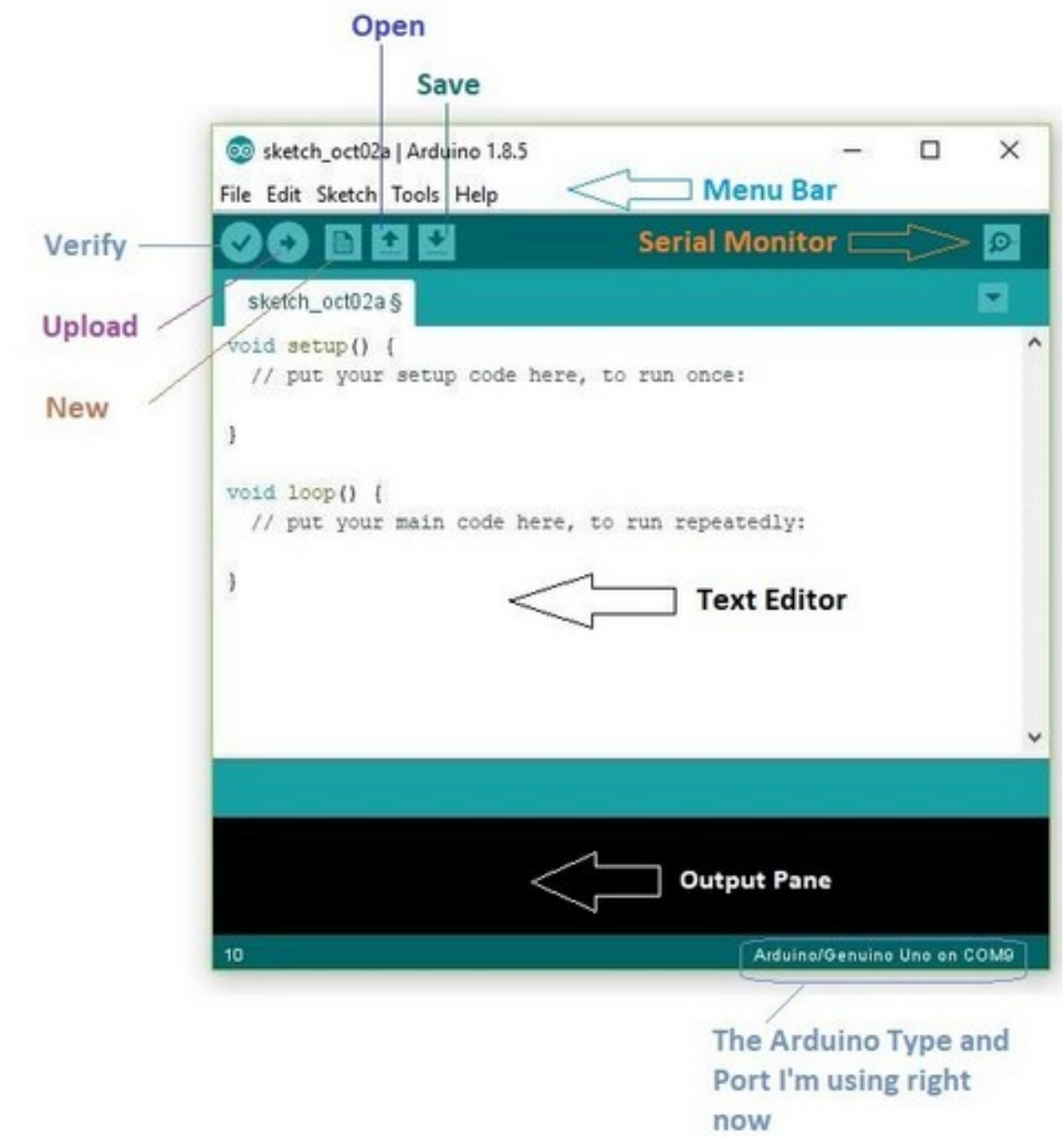

UNO


Reset Button
Resets the ATmega microcontroller
Pin 13 LED
The Arduino built-in LED
Digital Pins
Send and receive digital signals
USB Port
Used for powering Arduino, uploading sketches to Arduino, and for communicating with Arduino sketch
Power LED
This indicates that Arduino is receiving power
TX and RX LEDs
The following LEDs show that Arduino and computer are in communication
ATmega microcontroller
The heart and brain of Arduino Uno
Power connector
This is how you power your Arduino when it isn't plugged into a USB port for power. Acceptable voltages 7-12V dc.
Power Pins
Distribute power to inputs and outputs
Analog in
To receive an analog value
DIGITAL (PWM~)
UNO
ARDUINO
ON
ICSP
WWW.ARDUINO.CC – MADE IN ITALY
POWER
ANALOG IN
IOREF
RESET
3.3V
5V
GND
GND
Vin
A0
A1
A2
A3
A4
A5



Source: Fritzing

IDE (proprietária, grátis)



Programação

IDE Arduino

- Usa um dialeto C/C++
- Usa uma versão do GCC

Bibliotecas

- Grande quantidade de bibliotecas disponíveis
- Torna fácil utilizar um periférico novo (produzido para o Arduino)

Arduino Uno

- Mais difundido
- ATmega328p, 32 kB Flash, 2 kB RAM, 1 kB EEPROM
- 14 pinos de I/O, 6 entrada analógicas, 16 MHz

IDE

O programa é chamado de “Sketch”.

Pré-configurado com duas seções, Setup e Loop

- Setup roda uma única vez
- Loop fica rodando em loop no µC

Um pino I/O pode ser configurado como entrada ou saída

- pinMode(pino, modo)
  - pino varia de 0 a 13

digitalRead(pino)

- saída “1” (HIGH) para nível lógico alto

digitalWrite(pino, valor)

A tensão de saída HIGH depende da tensão de alimentação

LOW é ~ 0 V

```
radounsigned char val;
void setup()
{
    pinMode(13, OUTPUT);
    pinMode(7, INPUT);
    pinMode(9, INPUT_PULLUP);
}
void loop ()
{
    val = digitalRead(7);
    digitalWrite(13, HIGH);
    delay (1000);
    digitalWrite(13, LOW);
    delay (1000);
}
```

CCP (Capture/Compare/PWM)

Periférico do µC que permite ao programador controlar e medir o tempo de diversos eventos

Funcionamento está associado diretamente aos **temporizadores** e **contadores**, internos do µC, que são usados como sinais de entrada para o sistema

O ATmega328 possui 3 timers capazes de gerar até 6 sinais PWM nos pinos digitais nos pinos “~”

PWM é a sigla para Pulse Width Modulation, que consiste em um sinal digital com frequência fixa, no qual o tempo em que o sinal fica em nível alto ou baixo pode variar

O modo de captura permite a medição do tempo de duração de um determinado evento, monitorando o estado de um timer

O modo de comparação, compara o valor do timer com um valor previamente configurado, permitindo gerar ações quando o valor da contagem do temporizador igualar-se ao valor de referência

O modo PWM gera sinais modulados por largura de pulso, permitindo ao programador escolher a frequência e o ciclo de trabalho a serem adotados, com base nos sinais gerados pelo temporizador

Por serem partes da mesma estrutura, só é possível utilizar um sistema de cada vez

analogWrite (pino, valor)

- pino - onde será gerado o PWM
- valor - razão cíclica do PWM, valor entre 0 e 255

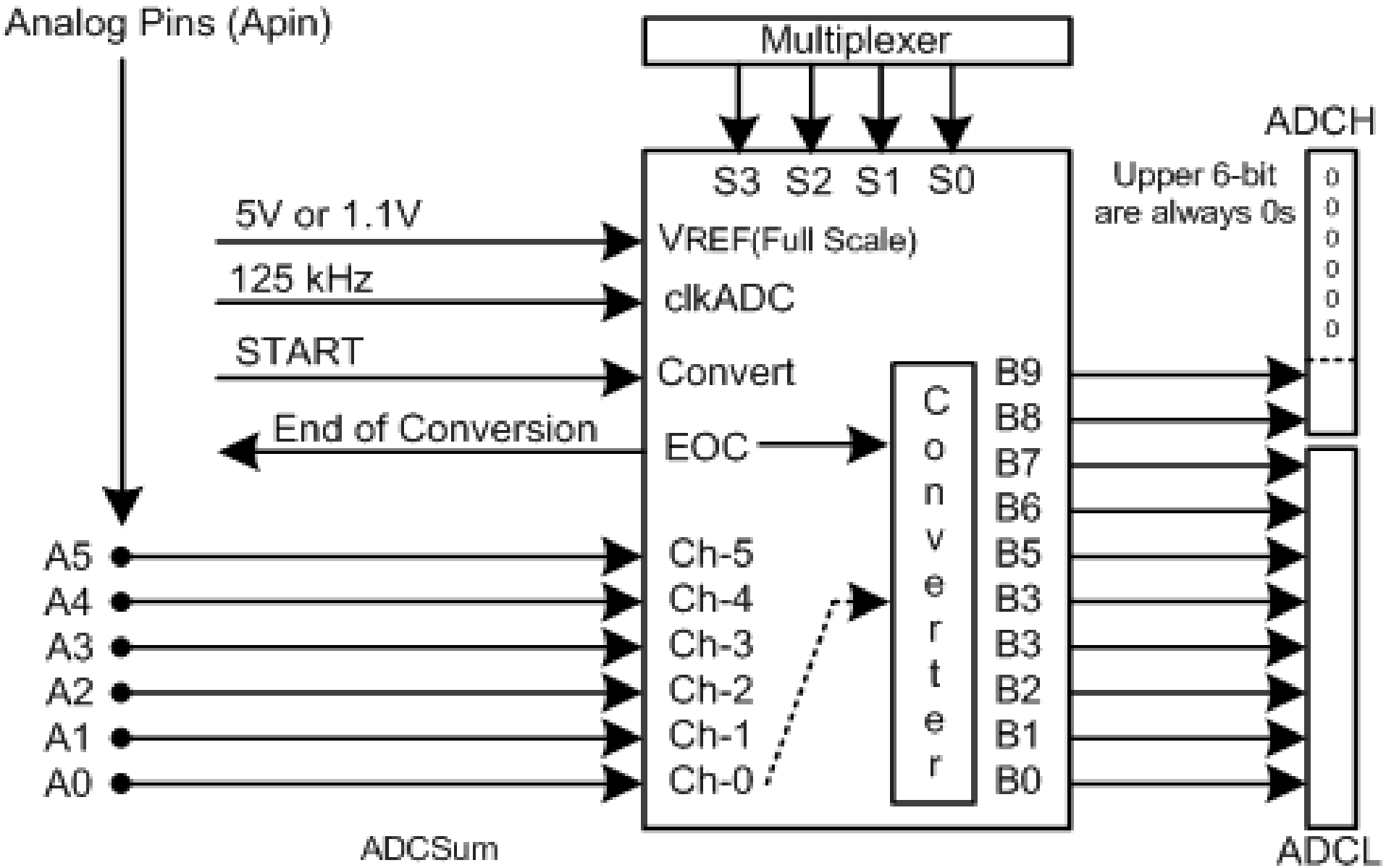
Alguns valores disponíveis no ambiente de desenvolvimento

- 980 Hz para os pinos 5 e 6
- 490 Hz para 3, 9, 10, 11

Conversor analógico-digital (A/D)

- ATmega328 possui um único, de 10 bits
- Pode ser multiplexado em 6 canais (`analogRead (pino)`)
  - Um único canal por vez, 0 a 5
- Retorna um número de 0 a 1023
- O intervalo de tensões depende da tensão de alimentação

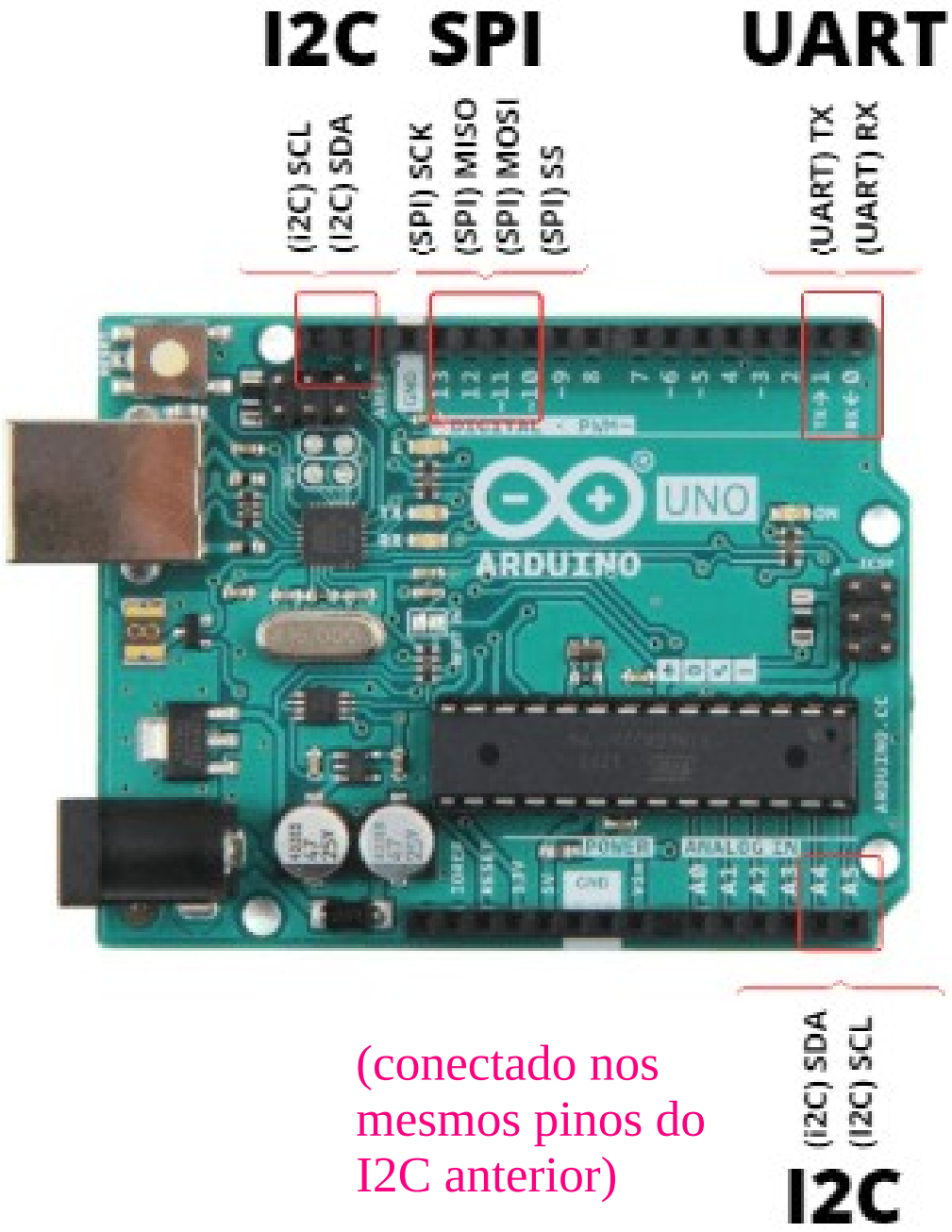
ADC Module inside ATmega328P MCU



As funções analógicas do Arduino não necessitam de inicialização

```
int PinoSensor = A2;
int PinoPWM = 5;
int ValorSensor;
void setup()
{
    // sem inicialização
}
void loop()
{
    ValorSensor = analogRead(PinoSensor);
    analogWrite(PinoPWM, ValorSensor/4);
    delay(10);
}
```

Comunicação



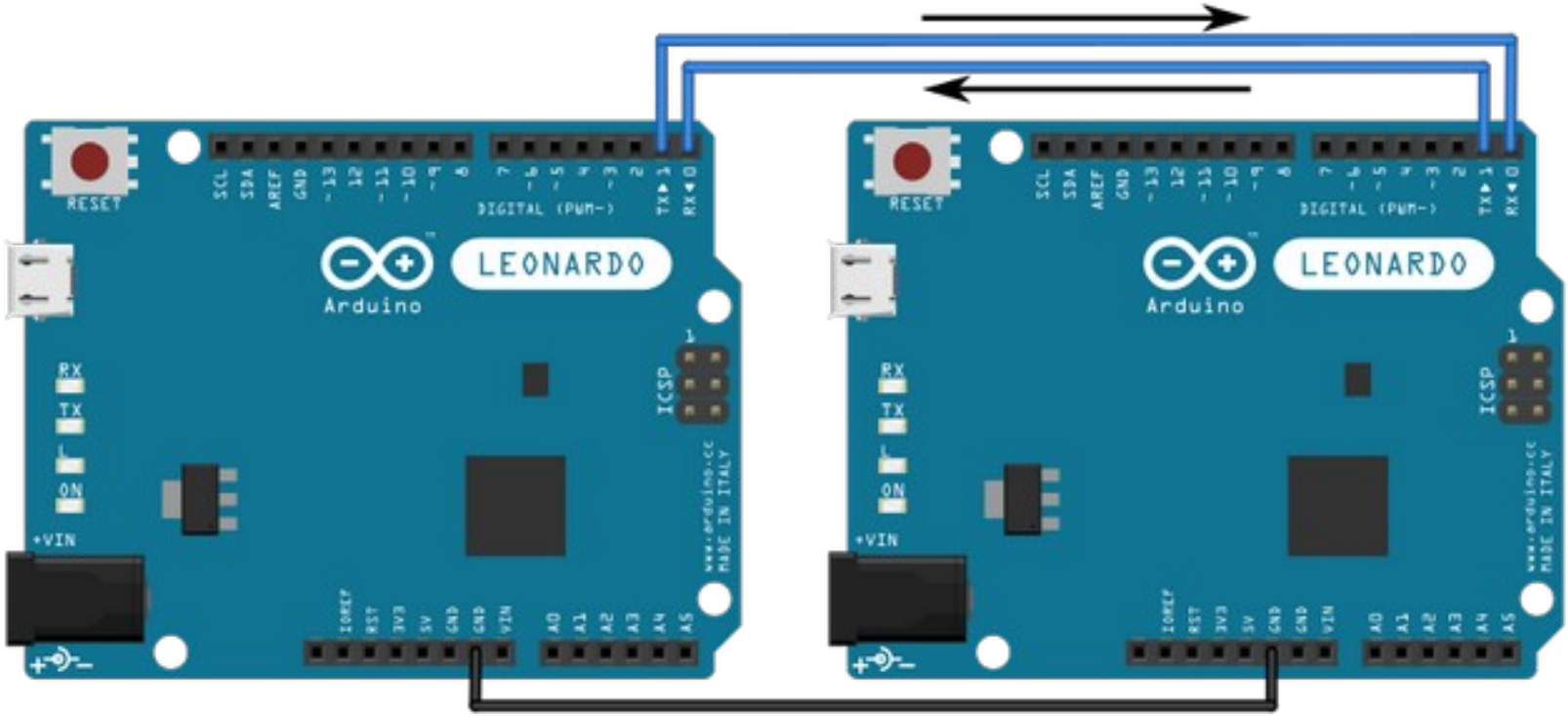
USB

(conectado nos mesmos pinos do I2C anterior)

UART

UART - Universal Asynchronous Serial Receiver Transmitter

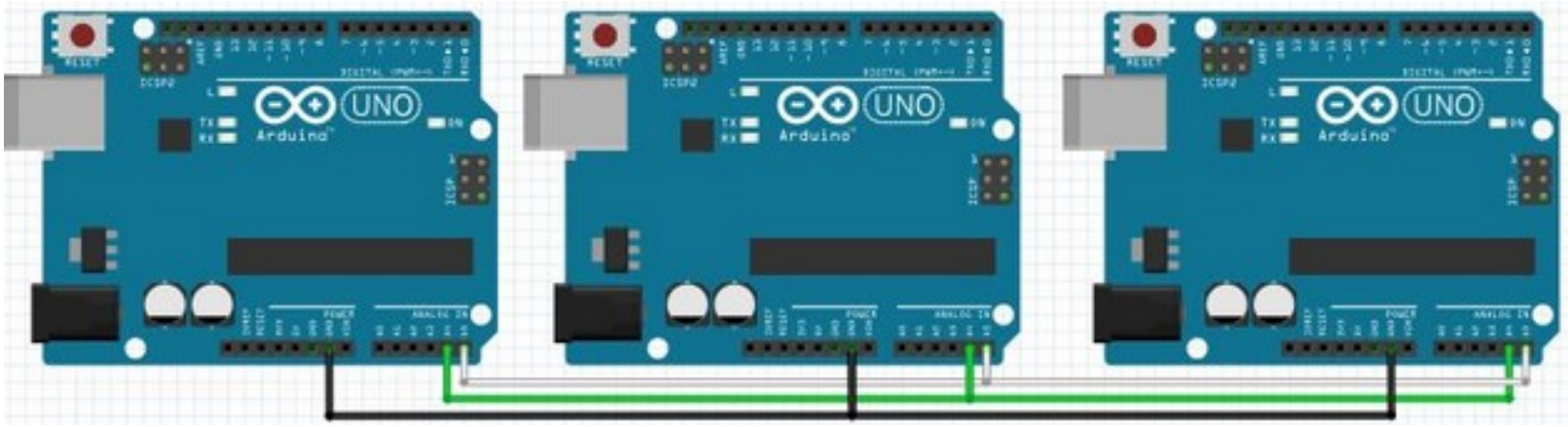
- 2 fios, o mais lento
- Apenas 2 dispositivos
  - Síncrono - clock, transmissão (pino 1), recepção (pino 0)
  - Assíncrono - terra, transmissão, recepção



I2C - Inter-Integrated Circuit

2 fios, mais rápido que UART, e mais lento que SPI

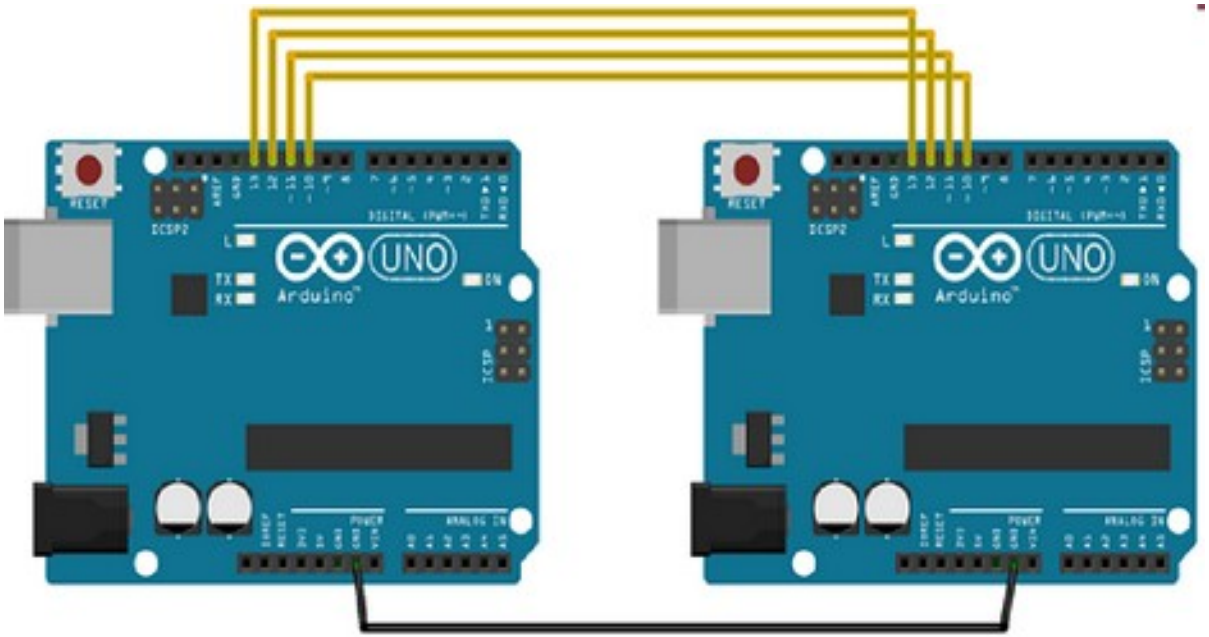
multi-master, multi-slave, até 128 devices



SPI - Serial Peripheral Interface

4 fios, ~ 4 Mbps slave, mais rápido que I2C

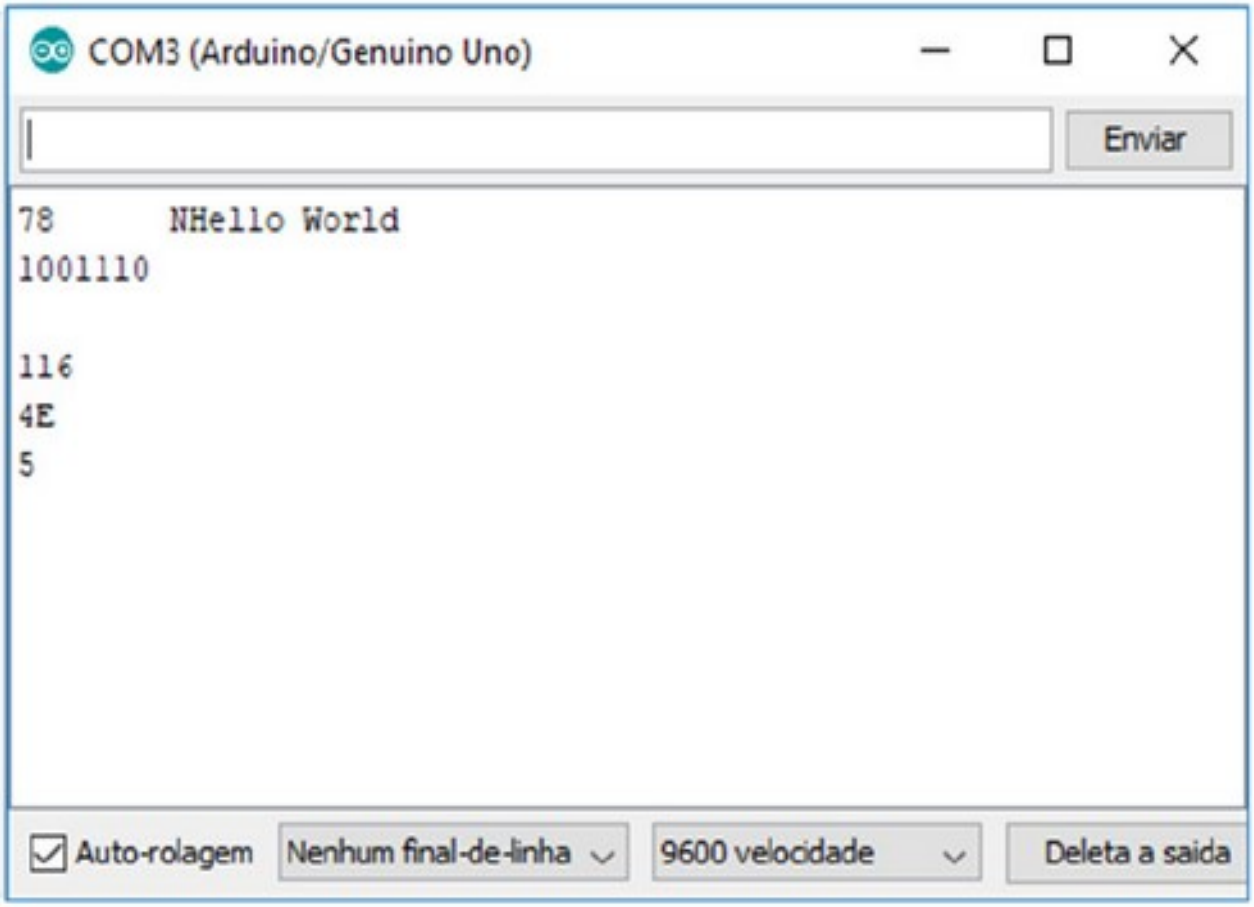
1 master, ~ 17 slaves no uno limitado a distâncias e capacitâncias



Exemplo

```
int a = 5;
void setup()
{
  Serial.begin(9600);

  Serial.print(78);
  Serial.print("\t");
  Serial.print('N');
  Serial.println("Hello World");
  Serial.println(78,BIN);
  Serial.println("");
  Serial.println(78,OCT);
  Serial.println(78,HEX);
  Serial.println(a);
}
```



Arduino mandando comunicação para o PC

PC recebendo a comunicação em um terminal